



Debre Berhan University
College of Computing
Department
of
Information Technology

Course Title: Advanced Programming

Course Code: ITec 3057

Credit : 3 Cr. Hrs / 5 ECTS (2 Hrs. Lec. & 3 Hrs. Lab)

Target group: 3rd year Information Technology

Course Manager: Yirga G.

Chapter 4:

Servlets and

Java Server Pages

Contents

- Definition of a Servlet
- Advantages of using servlets
- Life cycle of a servlet
- Generic Servlet and HttpServlet API
- Use a servlet to access a database
- Java Server Pages (JSP)

Definition of a Servlet

- A **servlet is a Java class that extends the functionality of a server**, particularly web servers, to handle requests and generate responses.
- Servlets are **typically used to create dynamic web pages and applications**.
- They run within a servlet container, which is part of the web server and manages the servlet's lifecycle and interaction with client requests.
- Properties of Servlets are as follows:
 - Servlets **work on the server-side**.
 - Servlets are **capable of handling complex requests obtained from the webserver**.

Servlet...

- Execution of Servlets basically involves six basic steps:
 1. The clients send the request to the webserver.
 2. The web server receives the request.
 3. The web server passes the request to the corresponding servlet.
 4. The servlet processes the request and generates the response in the form of output.
 5. The servlet sends the response back to the webserver.
 6. The web server sends the response back to the client and the client browser displays it on the screen.

Servlet...

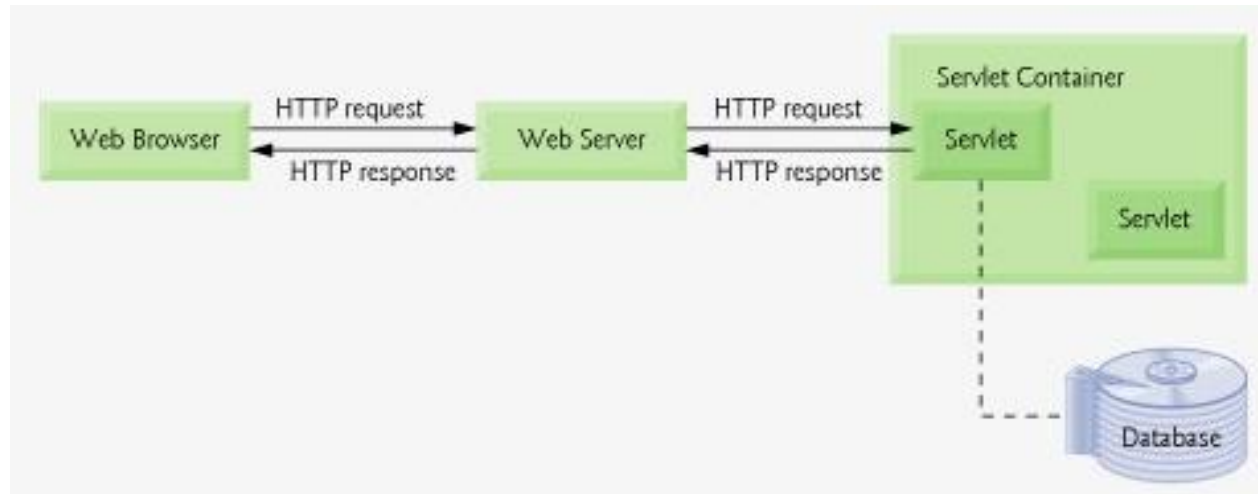
- Servlets are build from two packages:
 - **javax.servlet**
 - **javax.servlet.http**
- Packages **javax.servlet** and **javax.servlet.http** provide the classes and interfaces to define servlets.
- Packages `javax.servlet.jsp` and `javax.servlet.jsp.tagext` provide the classes and interfaces that extend the servlet capabilities for Java Server Pages.
- JSP allows Web-page implementers to create pages that encapsulate Java functionality and even to write scriptlets of actual Java code directly in the page.

Thin Clients

- Many developers feel that **servlets are the right solution for database-intensive applications** that communicate with so-called **thin clients applications**
- **Thin clients** applications provide presentation but do not process data, thus requiring fewer computing resources.
- The server is responsible for database access.
- Clients connect to the server using standard protocols available on most client platforms.
- The **presentation-logic code** for generating dynamic content can be written once and
 - reside on the server for access by clients
 - allow programmers to create efficient thin clients.

Servlet Architecture

- A **servlet** is a small Java program that runs within a Web server.
- **Servlets** receive and respond to requests from Web clients, usually across HTTP, the HyperText Transfer Protocol.



Advantages of a Java Servlet

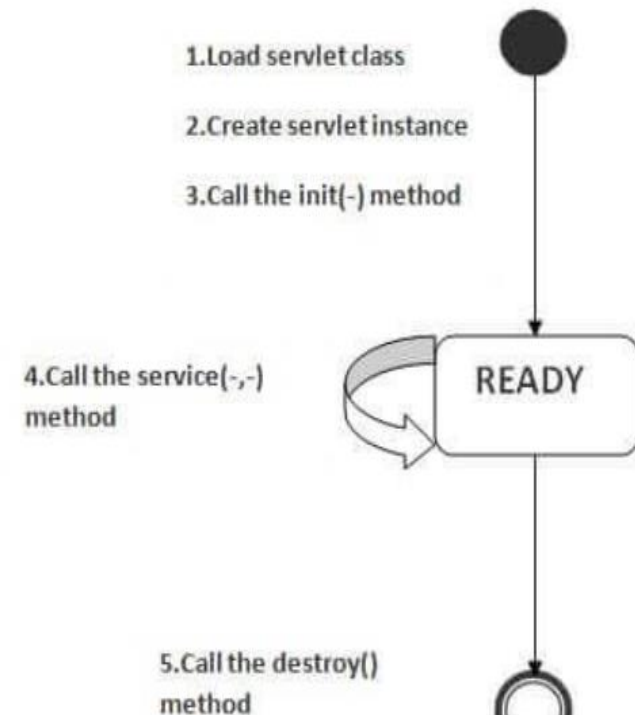
- Servlets, as written in Java, are platform-independent.
- Removes the overhead of creating a **new process** for each request as Servlet doesn't run in a separate process it makes servlet faster.
 - There is only a single instance that handles all requests concurrently.
 - This also saves the memory and allows a Servlet to easily manage the client state.
- It is a server-side component, so Servlet inherits the **security** provided by the Web server.
- The **API** designed for Java Servlet automatically acquires the advantages of the Java platforms such as platform-independent and portability.
 - In addition, it obviously can use the wide range of APIs created on Java platforms such as **JDBC** to access the database.
- Many Web servers that are suitable for personal use or low-traffic websites are offered for free or at extremely **cheap costs** eg. Java servlet.
 - However, the majority of commercial-grade Web servers are rather expensive, with the notable exception of Apache, which is free.

Interface Servlet and the Servlet Life Cycle

- Architecturally, **all servlets must implement the Servlet interface** of package javax.servlet.
- The methods of interface Servlet are invoked by the servlet container.
- A servlet life cycle can be defined as the entire process from its creation till the destruction.
- The following are the paths followed by a servlet
 - The servlet is initialized by calling the **init ()** method.
 - The servlet calls **service()** method to process a client's request.
 - The servlet is terminated by calling the **destroy()** method.
- Finally, servlet is garbage collected by the garbage collector of the JVM.

Servlet's life cycle ...

- The web container maintains the life cycle of a servlet instance.
- Let's see the life cycle of the servlet:
 1. Servlet class is loaded.
 2. Servlet instance is created.
 3. `init()` method is invoked.
 4. `service()` method is invoked.
 5. `destroy()` method is invoked.



Servlet's life cycle ...

1) Servlet class is loaded

- The classloader is responsible to load the servlet class. The servlet class is loaded when the first request for the servlet is received by the web container.

2) Servlet instance is created

- The web container creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.

3) init method is invoked

- The web container calls the init method only once after creating the servlet instance. The init method is used initialize the servlet. It is the life cycle method of the javax.servlet.Servlet interface. Syntax of the init method given below:
 - 1. `public void init(ServletConfig config) throws ServletException`

4) service method is invoked

- The web container calls the service method each time when request for the servlet is received. If servlet is not initialized, it follows the first three steps as described above then calls the service method. If servlet is initialized, it calls the service method. Notice that servlet is initialized only once.
- The syntax of the service method of the Servlet interface is given below:
 - 1. `public void service(ServletRequest request, ServletResponse response) throws ServletException, IOException`

5) destroy method is invoked

- The web container calls the destroy method before removing the servlet instance from the service. It gives the servlet an opportunity to clean up any resource for example memory, thread etc. The syntax of the destroy method of the Servlet interface is given below:
 - `public void destroy()`

Servlet's life cycle ...

- The servlet packages define two abstract classes that implement interface `Servlet`
 - class **GenericServlet** (from the package `javax.servlet`) and
 - class **HttpServlet** (from the package `javax.servlet.http`).
- `GenericServlet` is a protocol-independent servlet.
- `HttpServlet` uses the HTTP protocol to exchange the information between the server and the client.
- The key method in every servlet is `service`, which accepts both a `ServletRequest` object and a `ServletResponse` object.
- These objects provide access to input and output streams that allow the servlet to read data from and send data to the client.
- If problems occur during the execution of a servlet, either `ServletExceptions` or `IOExceptions` are thrown to indicate the problem.

HttpServlet Class

- Servlets typically extend class HttpServlet, which **overrides method service** to distinguish between the typical requests received from a client Web browser.
- The two most common **HTTP request types** (also known as **request methods**) are **get** and **post**.
- Class HttpServlet **defines methods doGet and doPost** to respond to get and post requests from a client,
- Methods doGet and doPost accept as arguments and HttpServletRequest object and an HttpServletResponse object that enable interaction between the client and the server.
- The methods of **HttpServletRequest allow access** to the data supplied as part of the request.
 - enables a servlet to obtain information about a client request.
- The **HttpServletResponse methods make it easy** to return the servlet's results to the Web client.
 - enables a servlet to formulate an HTTP response to a client.

HttpServletRequest Interface

- Every call to **doGet** or **doPost** for an **HttpServletRequest** **receives** an object that implements interface **HttpServletRequest**
- The servlet container creates an **HttpServletRequest** object and passes it to the servlet's service method
 - Which, in turn, passes it to **doGet** or **doPost**.
- This object contains the client's request and provides methods that enable the servlet to process the request.
- Some methods are from interface **ServletRequest** the interface that **HttpServletRequest** extends.

HttpServletResponse Interface

- Every call to **doGet** or **doPost** for an **HttpServlet** **receives an object** that implements interface **HttpServletResponse**.
- The servlet container creates an **HttpServletResponse object** and **passes it to the servlet's service method**
 - Which, in turn, passes it to **doGet** or **doPost**.
- This object provides methods that enable the servlet to formulate the response to the client.
- Some of these are from interface **ServletResponse** the interface that **HttpServletResponse** extends.

Handling HTTP get Requests

- The **primary purpose of an HTTP get request is to retrieve the content of a specified URL** normally an HTML or XHTML document.
- Here we will develop a servlet that handles an HTTP GET request. The servlet is invoked when a form on a web page is submitted.
- The example contains two files.
- A web page is defined in **ColorGet.html**, and a servlet is defined in **ColorGetServlet.java**. The HTML source code for **ColorGet.html** is shown in the following listing.
- It defines a form that contains a select element and a submit button. Notice that the action parameter of the form tag specifies a URL.
- The URL identifies a servlet to process the HTTP GET request.

Handling HTTP get Requests

- **//ColorGet.html**
- `<html>`
- `<body>`
- `<center>`
- `<form name="Form1"`
`action="http://localhost:8080/examples/servlets/servlet/ColorGetServ`
`let">`
- `<B>Color:</B>`
- `<select name="color" size="1">`
- `<option value="Red">Red</option>`
- `<option value="Green">Green</option>`
- `<option value="Blue">Blue</option> </select>`
- `<br><br>`
- `<input type=submit value="Submit"> </form>`
- `</body>`
- `</html>`

Handling HTTP get Requests

- **//ColorGetServlet.java**
- `import java.io.*;`
- `import javax.servlet.*;`
- `import javax.servlet.http.*;`
- `public class ColorGetServlet extends HttpServlet {`
 - `public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {`
 - `String color = request.getParameter("color");`
 - `response.setContentType("text/html");`
 - `PrintWriter pw = response.getWriter();`
 - `pw.println("<B>The selected color is: ");`
 - `pw.println(color);`
 - `pw.close();`
 - `}`
- `}`

Handling HTTP get Requests

- Compile the servlet.
- Next, copy it to the appropriate directory, and update the **web.xml** file, as previously described.
- Then, perform these steps to test this example:
 - Start Tomcat, if it is not already running.
 - Display the web page in a browser.
 - Select a color.
 - Submit the web page.

Handling HTTP post Requests

- An HTTP post request is frequently used to send data from an HTML form to a server-side form handler that processes the data.
- Here we will develop a servlet that handles an HTTP POST request.
- The servlet is invoked when a form on a web page is submitted.
- The example contains two files.
 - A web page is defined in **ColorPost.html**, and a servlet is defined in **ColorPostServlet.java**.
 - The HTML source code for **ColorPost.html** is shown in the following listing.
- It is identical to **ColorGet.html** except that the method parameter for the form tag explicitly specifies that the **POST method should be used**, and the action parameter for the form tag specifies a different servlet.

Handling HTTP post Requests

- **//ColorGet.html**
- `<html>`
- `<body>`
- `<center>`
- `<form name="Form1" method="post"
action="http://localhost:8080/examples/servlets/servlet/ColorPostSer
vlet">`
- `<B>Color:</B>`
- `<select name="color" size="1">`
- `<option value="Red">Red</option>`
- `<option value="Green">Green</option>`
- `<option value="Blue">Blue</option> </select>`
- `<br><br>`
- `<input type=submit value="Submit"> </form>`
- `</body>`
- `</html>`

Handling HTTP post Requests

- **//ColorPostServlet.java**
- `import java.io.*;`
- `import javax.servlet.*;`
- `import javax.servlet.http.*;`
- `public class ColorPostServlet extends HttpServlet {`
- `public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {`
- `String color = request.getParameter("color");`
- `response.setContentType("text/html");`
- `PrintWriter pw = response.getWriter();`
- `pw.println("<B>The selected color is: ");`
- `pw.println(color);`
- `pw.close();`
- `}`
- `}`

Multitier Applications

- Servlets can communicate with databases via JDBC.
- Many of today's applications are **three-tier distributed applications**, consisting of
 - a **user interface/Presentation** tier,
 - **business logic tier** and
 - a **database access tier**/Layer.
- In multitier architectures,
 - Web servers often are used in the middle tier.
- Server-side components, such as servlets, execute in an application server alongside the Web server.
- These components provide the business logic that manipulates data from databases and communicates with client Web browsers.
- Servlets, through JDBC, can interact with popular database systems.
- Developers use SQL for queries, and JDBC drivers handle the specifics of interacting with each database system.

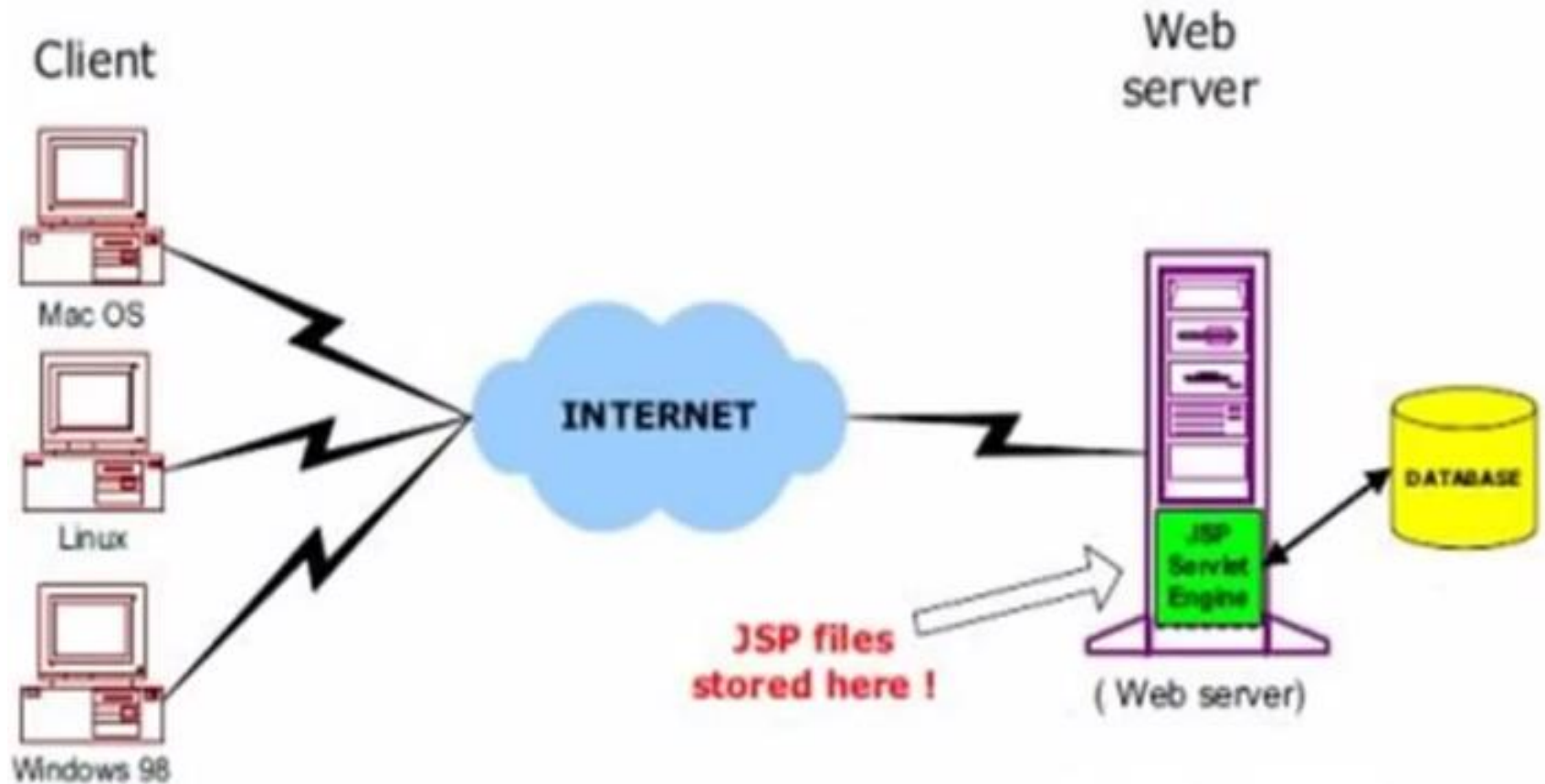
Java Server Pages

- **Java Server Pages (JSP)** an extension of servlet technology that **separates the presentation from the business logic**.
- Java Server Pages simplify the delivery of dynamic Web content.
- They enable Web application programmers to create dynamic content by reusing predefined components and by **interacting with components using server-side scripting**.
- Using JSP, you can collect input from users through web page forms, present records from a database or another source, and create web pages dynamically.
- It is more convenient to write (and to modify!) regular HTML than to have plenty of `println` statements that generate the HTML.

Java Server Pages

- Java Server Page programmers can use special software components called JavaBeans and custom tag libraries that encapsulate complex, dynamic functionality.
- Web sites use these custom tags like any other Web page element to take advantage of the more complex functionality hidden by the tag.
- The classes and interfaces that are specific to Java Server Pages programming are located in packages **javax.servlet.jsp** and **javax.servlet.jsp.tagext**.

JSP Architecture



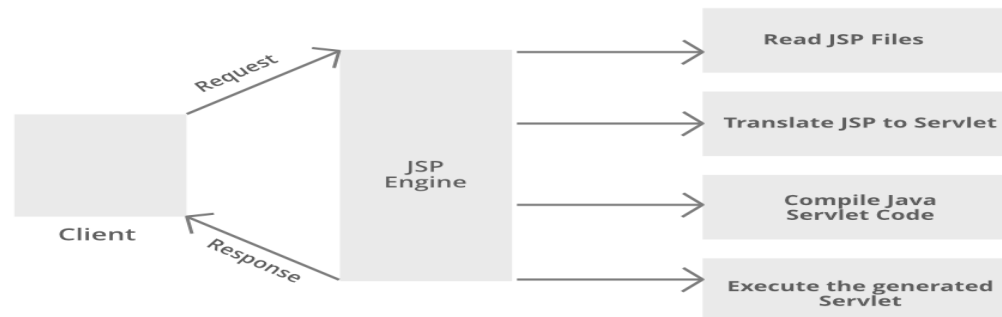
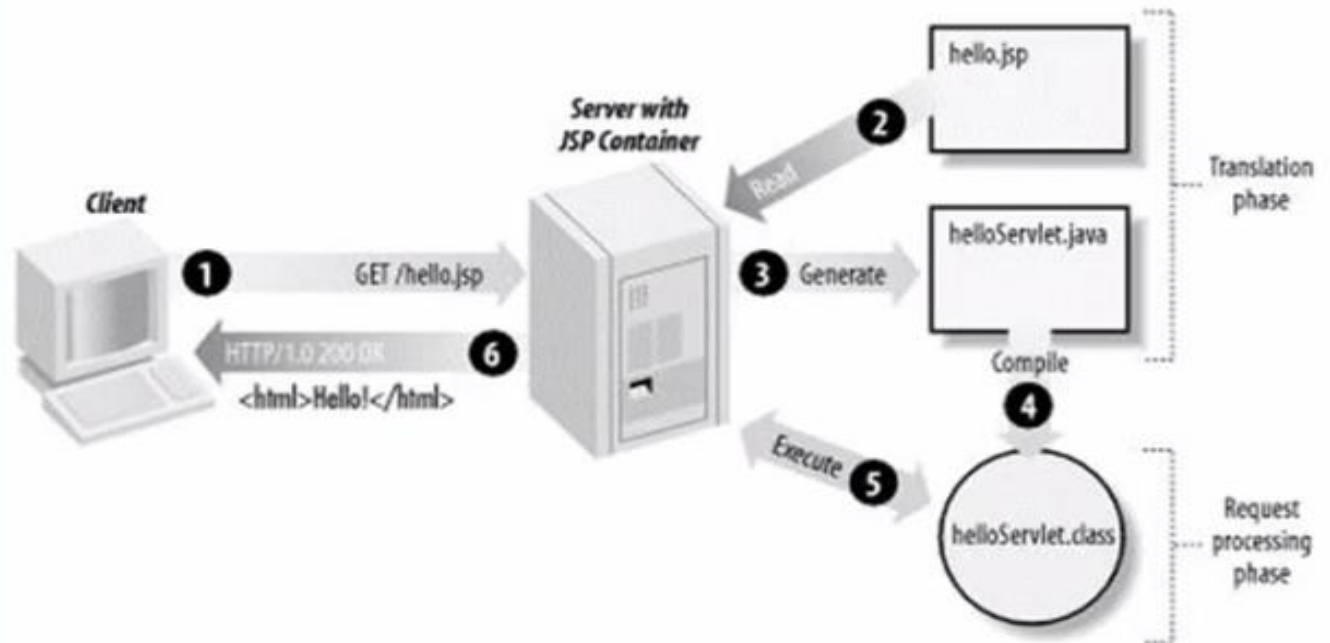
JSP Processing

- The following steps explain how the web server creates the web page using JSP:
 - As with a normal page, your browser sends an HTTP request to the web server.
 - The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with **.jsp** instead of **.html**.
 - The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to `println( )` statements and all JSP elements are converted to Java code that implements the corresponding dynamic behavior of the page.
 - The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
 - A part of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format, which the servlet engine passes to the web server inside an HTTP response.
 - The web server forwards the HTTP response to your browser in terms of static HTML content.
 - Finally web browser handles the dynamically generated HTML page inside the HTTP response exactly as if it were a static page.

JSP Processing

The browser sends an HTTP request to the web server.

The web server recognizes that the HTTP request is for a JSP page, because it sees the .jsp extension, and forwards it to a JSP container.



JSP ...

The following is an example of a JSP.

It contains a mix of HTML and Java code where the Java code handles all the page logic and the HTML manages the display.

```
<%@page import="java.util.Date" %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Today's Date</title>
  </head>
  <%
    Date date= new Date();
  %>
  <body>
    <p>Today's date is <%= date %> </p>
  </body>
</html>
```

The text between `<%` and `%>` is our Java code which, when executed, assigns the variable `date` the current system date and then writes the value of the variable to the HTML.

JSP Components

- There are four key components to JSPs: **directives**, **actions**, **scripting elements** and **tag libraries**.
 - **Directives** are messages to the JSP container the server component that executes JSPs that enable the programmer to specify page settings, to include content from other resources and to specify custom tag libraries for use in a **JSP**.
 - **Actions** encapsulate functionality in predefined tags that programmers can embed in a JSP.
 - Actions often are performed based on the information sent to the server as part of a particular client request.
 - They also can create Java objects for use in JSP scriptlets.
 - **Scripting elements** enable programmers to insert Java code that interacts with components in a JSP (and possibly other Web application components) to perform request processing.
 - Scriptlets, one kind of scripting element, contain code fragments that describe the action to be performed in response to a user request.
 - **Tag libraries** are part of the tag extension mechanism that enables programmers to create custom tags.
 - Such tags enable Web page designers to manipulate JSP content without prior Java knowledge.

JSP

- In some ways, Java Server Pages look like standard XHTML or XML documents.
- In fact, JSPs normally include XHTML or XML markup.
- Such markup is known as **fixed-template data** or **fixed-template text**.
- Fixed-template data often helps a programmer decide whether to use a servlet or a JSP.
- Programmers tend to use JSPs **when most of the content sent to the client is fixed-template data** and little or none of the content is generated dynamically with Java code.
- Programmers typically **use servlets when only a small portion of the content sent** to the client is **fixed-template data**.
- In fact, some servlets do not produce content.
- Rather, they perform a task on behalf of the client, then invoke other servlets or JSPs to provide a response.
- Note that in most cases servlet and JSP technologies are interchangeable.
- As with servlets, JSPs normally execute as part of a Web server.

JSP.....

- When a JSP-enabled server receives the first request for a JSP,
 - the JSP container translates the JSP into a Java servlet that handles the current request and future requests to the JSP.
- Literal text in a JSP becomes string literals in the servlet that represents the translated JSP.
- Any **errors that occur in compiling the new servlet** result in **translation-time errors**.
- The JSP container places the Java statements that implement the JSP's response in method **_jspService** at translation time.
- If the new servlet compiles properly, the JSP container invokes method **_jspService** to process the request.
- The JSP may respond directly or may invoke other Web application components to assist in processing the request.
- Any **errors that occur during request processing** are known as **request-time errors**.

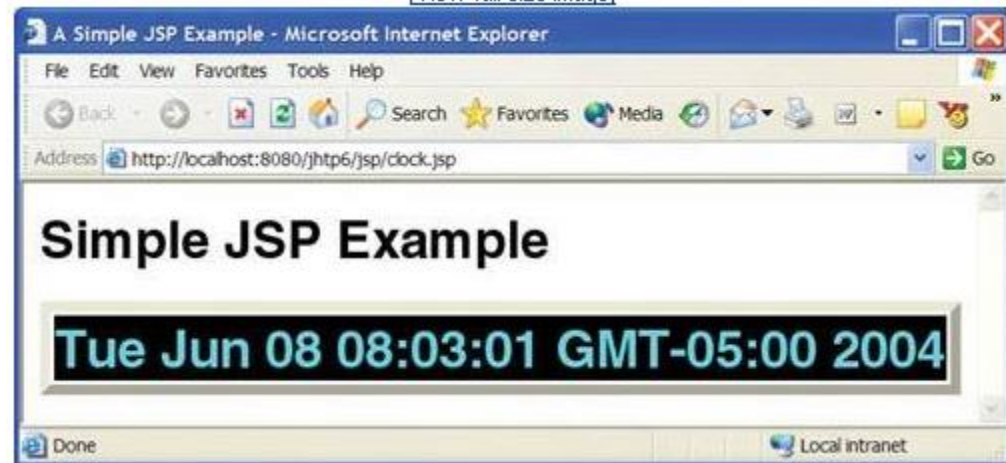
JSP.....

- Overall, the request-response mechanism and the JSP life cycle are the same as those of a servlet.
- JSPs can override methods **jspInit** and **jspDestroy** (similar to servlet methods `init` and `destroy`),
 - **jspInit** - the JSP container invokes when initializing
 - **jspDestroy** - the JSP container invokes when terminating a JSP
- JSP programmers can define these methods using JSP declarations part of the JSP scripting mechanism.

First JSP Example

- // clock.jsp
- // JSP expression inserting the date and time into a Web page.

```
1 <?xml version = "1.0"?>
2 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
3   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
4
5 <!-- clock.jsp -->
6
7 <html xmlns = "http://www.w3.org/1999/xhtml">
8   <head>
9     <meta http-equiv = "refresh" content = "60" />
10    <title>A Simple JSP Example</title>
11    <style type = "text/css">
12      .big { font-family: helvetica, arial, sans-serif;
13        font-weight: bold;
14        font-size: 2em; }
15    </style>
16  </head>
17  <body>
18    <p class = "big">Simple JSP Example</p>
19    <table style = "border: 6px outset;">
20      <tr>
21        <td style = "background-color: black;">
22          <p class = "big" style = "color: cyan;">
23            <!-- JSP expression to insert date/time -->
24            <%= new java.util.Date() %>
25          </p>
26        </td>
27      </tr>
28    </table>
29  </body>
30 </html>
```



Implicit Objects

- **Implicit objects** provide access to many servlet capabilities in the context of a Java Server Page.
- Implicit objects have four scopes:
 - application,
 - page,
 - request and
 - session.
- The JSP container owns objects with application scope.
- Any JSP can manipulate such objects.
- Objects with **page scope** exist only in the page that defines them.
- Each page has its own instances of the page-scope implicit objects.
- Objects with **request scope** exist for the duration of the request.
- For example, a JSP can partially process a request, then forward it to a servlet or another JSP for further processing.
- Request-scope objects go out of scope when request processing completes with a response to the client.
- Objects with session scope exist for the client's entire browsing session.

Scripting

- Java Server Pages often present dynamically generated content as part of an XHTML document that is sent to the client in response to a request.
- In some cases, the content is static but is output only if certain conditions are met during a request (e.g., providing values in a form that submits a request).
- JSP programmers can insert Java code and logic in a JSP using scripting.

Scripting Components

- The JSP scripting components include
 - scriptlets,
 - comments,
 - expressions,
 - declarations and
 - escape sequences.
- **Scriptlets**
 - are blocks of code delimited by `<%` and `%>`.
 - contain Java statements that the container places in method **`_jspService`** at translation time.
- **Comments**
 - JSPs support three comment styles:
 - JSP comments,
 - XHTML comments and
 - scripting-language comments.

Scripting Components ...

- JSP comments
 - are delimited by `<%--` and `--%>`.
 - These can be placed throughout a JSP, but not inside scriptlets.
- XHTML **comments**
 - are delimited with `<!--` and `-->`.
 - These, too, can be placed throughout a JSP, but not inside scriptlets.
- Scripting language comments
 - are currently Java comments, because Java currently is the only JSP scripting language.
 - Scriptlets can use Java's end-of-line `//` comments and traditional comments (delimited by `/*` and `*/`).
- JSP comments and scripting-language comments are ignored and do not appear in the response to a client.
- When clients view the source code of a JSP response, they will see only the XHTML comments in the source code.
- The different comment styles are useful for separating comments that the user should be able to see from those that document logic processed on the server.

Scripting Components

- JSP expressions
 - are delimited by `<%=` and `%>`
 - contain a Java expression that is evaluated when a client requests the JSP containing the expression.
 - The container converts the result of a JSP expression to a String object, then outputs the String as part of the response to the client.
- Declarations
 - delimited by `<%!` and `%>`
 - enable a JSP programmer to define variables and methods for use in a JSP.
 - Variables become instance variables of the servlet class that represents the translated JSP.
 - Similarly, methods become members of the class that represents the translated JSP.
 - Declarations of variables and methods in a JSP use Java syntax.
 - Thus, a variable declaration must end with a semicolon, as in
`<%! int counter = 0; %>`
- JSP escape sequences
 - Special characters or character sequences that the JSP container normally uses to delimit JSP code can be included in a JSP as
 - literal characters in scripting elements,
 - fixed template data and
 - attribute values using escape sequences.

Scripting Example

- `//Scripting a JavaServer Pagewelcome.jsp.`
- `<?xml version = "1.0" ?>`
- `<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"`
`"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">`
- `<!-- welcome.jsp -->`
- `<!-- JSP that processes a "get" request containing data. -->`
- `<html xmlns = "http://www.w3.org/1999/xhtml">`
- `<!-- head section of document -->`
- `<head>`
- `<title>Processing "get" requests with data</title>`
- `</head>`
- `<!-- body section of document -->`
- `<body>`
- `<% // begin scriptlet`
- `String name = request.getParameter( "firstName" );`
- `if ( name != null )`
- `{`
- `%>`
- `<%-- end scriptlet to insert fixed template data --%>`
- `<h1>`
`Hello <%= name %>, <br />`
`Welcome to JavaServer Pages!`
- `</h1>`
- `<% // continue scriptlet`
- `} // end if`

Scripting Example

- `else {`
- `%>`
- `<%-- end scriptlet to insert fixed template data --%>`
- `<form action = "welcome.jsp" method = "get">`
- `<p>Type your first name and press Submit</p>`
- `<p><input type = "text" name = "firstName" />`
- `<input type = "submit" value = "Submit" />`
- `</p>`
- `</form>`
- `<% // continue scriptlet`
- `} // end else`
- `%>`
- `<%-- end scriptlet --%>`
- `</body>`
- `</html> <!-- end XHTML document -->`

Scripting Example



Question?